Operating Systems — Study Guide

Priority Topics 1, 2 and 3: Process Management, CPU Scheduling, Synchronization & Deadlocks

This guide is based on the priority topics identified from the two uploaded E3 question papers. Explanations and practice questions are written for revision; practice questions are not copied verbatim from the papers.

1. Process Management

A process is a program in execution. It includes code and execution context such as the program counter, CPU registers, stack, address space and resources managed by the operating system.

1.1 Process Control Block (PCB)

The PCB is an OS data structure that stores information needed to manage a process:

• **Process state:** new, ready, running, waiting/blocked or terminated.

• **PID and PPID:** process identifier and parent process identifier.

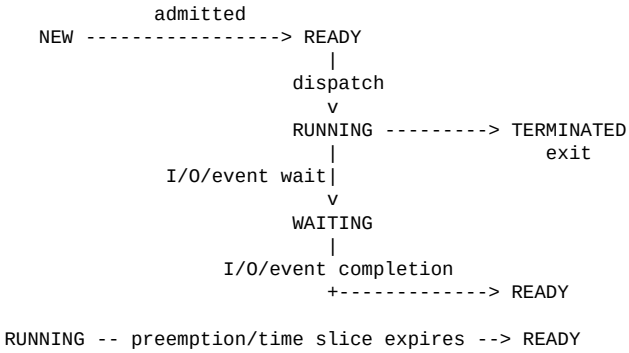
• **CPU context:** program counter and saved register values.

• **Scheduling data:** priority and scheduling-queue information.

• **Memory details:** address-space and memory-management information.

• **I/O and accounting:** open-resource references and usage information.

1.2 Process states




```
RUNNING -- preemption/time slice expires --> READY
```

Remember: a blocked process normally becomes ready when its event completes; a preempted running process normally returns to ready.

1.3 Process identifiers

• **PID:** identifies the process.

• **PPID:** identifies the parent process.

• Linux getpid() returns the current PID; getppid() returns the parent PID.

1.4 fork(), exec(), wait(), exit()

• **fork():** creates a child process. On success, returns 0 in the child and the child's PID in the parent; on failure, a negative value.

• **exec family:** replaces the current process image with a new program. Successful exec does not return to the old code.

• **wait()/waitpid():** lets the parent wait for a child and collect its termination status.

• **exit():** terminates a process and supplies an exit status. Use WIFEXITED(status) and WEXITSTATUS(status) to inspect normal termination and its status.

1.5 Zombie and orphan processes

• **Zombie:** a child has terminated, but the parent has not yet collected its status; a process-table entry remains temporarily.

• **Orphan:** a child continues after its parent terminates and is re-parented by the system.

• Parents should collect child status with wait functions and handle child lifecycles deliberately.

1.6 Context switching

A context switch occurs when the CPU stops running one process/thread and switches to another. The OS saves the outgoing execution context and restores the incoming context. This adds overhead because the CPU is doing management work rather than application work during the switch.

1.7 Process vs thread

Aspect	Process	Thread
Address space	Normally separate from other processes	Threads in a process share the address space
Resources	Own process-level resources	Shares many process resources; own stack and execution context
Communication	Often uses IPC	Can use shared memory; synchronization is needed
Overhead	Often more costly to create/switch	Often lighter-weight, but shared data needs coordination

Topic 1 checklist: explain PCB and process states; trace fork return values; distinguish exec/wait/exit; compare zombie and orphan; describe context switching.

2. CPU Scheduling

CPU scheduling chooses a ready process to run. Common goals include efficient CPU use, good throughput, and reasonable response, waiting and turnaround times.

2.1 Key terms and formulas

• **Arrival time (AT):** when a process enters the ready queue.

• **Burst time (BT):** CPU time required.

• **Completion time (CT):** when the process finishes.

• **Turnaround time (TAT) = CT** – **AT.**

• **Waiting time (WT) = TAT** – **BT** in the simple single-CPU-burst model.

• **Response time:** first CPU start time minus arrival time.

• **Throughput:** completed processes per unit time.

• **Dispatch latency:** time to stop one process and start another.

2.2 Scheduling algorithms

Algorithm	How it works	Key point
FCFS	Runs in arrival order	Simple; may cause convoy effect
SJF	Chooses shortest next CPU burst	Can reduce average waiting; long jobs may starve
Priority	Runs highest-priority ready process	Starvation possible; aging can help
Round Robin	Each process gets up to a time quantum	Preemptive; useful for time-sharing

2.3 Preemptive vs non-preemptive

Non-preemptive: once scheduled, a process keeps the CPU until it finishes or blocks. FCFS and basic SJF are commonly taught as non-preemptive variants.

Preemptive: the OS can take the CPU away, for example when a time slice expires. Round Robin is preemptive; Shortest Remaining Time First is the preemptive SJF variant.

2.4 Worked example: non-preemptive SJF

Assume all processes arrive at time 0:

Process	Burst (ms)	SJF order
P1	6	3
P2	8	4
P3	7	2
P4	3	1

Order: P4 → P1 → P3 → P2

```
Gantt chart: 0 | P4 | 3 | P1 | 9 | P3 | 16 | P2 | 24
```

Waiting times: P4 = 0 ms; P1 = 3 ms; P3 = 9 ms; P2 = 16 ms. Average waiting time = (0 + 3 + 9 + 16) / 4 = **7 ms**.

2.5 What to remember

• Round Robin is preemptive.

• Priority scheduling can cause starvation; aging gradually improves a waiting process's priority.

• A very small time quantum increases context-switch overhead; a very large quantum makes Round Robin resemble FCFS.

• Check arrival times before calculating the schedule.

Topic 2 checklist: know WT/TAT formulas; draw Gantt charts; calculate CT, TAT and WT; classify algorithms; explain starvation, aging and dispatch latency.

3. Synchronization and Deadlocks

Concurrent processes or threads may access shared state during overlapping periods. Without coordination, the result can depend on the order in which operations occur.

3.1 Race condition and critical section

A **race condition** occurs when the result depends on the timing/order of concurrent operations. A **critical section** is code that accesses shared data or a resource and needs protection when concurrent access could cause errors.

Three common requirements for a critical-section solution:

• **Mutual exclusion:** at most one participant enters the protected critical section at a time.

• **Progress:** if the critical section is free, choosing the next entrant should not be postponed indefinitely by processes that are not trying to enter.

• **Bounded waiting:** a requester should not wait forever while others repeatedly enter.

3.2 Semaphores

A semaphore is a synchronization variable operated on atomically, commonly using **wait (P/down)** and **signal (V/up)**. Wait may block when the resource/condition is unavailable; signal makes a resource or condition available.

```
wait(S);       // acquire / wait
// critical section
signal(S);     // release / notify
```

Some textbook conventions use negative semaphore values to indicate waiting processes. The exact interpretation depends on the semaphore definition; many modern APIs do not expose a negative count.

3.3 Mutex and atomic operations

A mutex is a mutual-exclusion lock. A thread locks it before a protected region and unlocks it afterward. Locking a non-recursive mutex already owned by the same thread can block indefinitely (self-deadlock).

Atomic hardware primitives such as Test-and-Set perform an indivisible operation and can be used to build locks. Atomicity of one operation does not automatically make a multi-step algorithm correct.

3.4 Deadlock and starvation

Deadlock: a set of processes/threads is stuck because each waits for a resource/event that cannot become available due to a circular dependency.

Starvation: a process waits indefinitely or unacceptably long because resources or CPU time keep going to others. Starvation does not require circular wait.

Four classic necessary conditions for deadlock:

• **Mutual exclusion:** at least one resource is non-shareable.

• **Hold and wait:** a process holds one resource while waiting for another.

• **No preemption:** resources cannot simply be taken away; they must be released voluntarily.

• **Circular wait:** a cycle exists in the resource-waiting chain.

3.5 Semaphore-tracing questions

For a problem with P0 repeatedly waiting on S0, printing, then signalling S1 and S2, while P1 waits on S1 and signals S0 and P2 waits on S2 and signals S0, track each process's current instruction and each semaphore's state. If P0 loops, P1 and P2 can each enable further iterations. The exact print count depends on the full loop and scheduling constraints; do not assume a fixed count without tracing all possible steps.

3.6 Practical rules

• Keep critical sections short.

• Acquire multiple locks in a consistent order to reduce circular-wait risk.

• Release locks on all error/return paths.

• Use mutexes for mutual exclusion and semaphores for counts/events when appropriate.

• Use logs and race-detection tools to diagnose races and hangs.

Topic 3 checklist: define race condition and critical section; explain semaphore wait/signal; distinguish deadlock from starvation; list four deadlock conditions; explain atomic operations.

Practice Questions

Choose one answer for each question, then check the key below.

1. What does fork() return in the child?

A. Child PID

B. 0

C. -1

D. Parent PID

2. Which structure stores process management information?

A. PCB

B. ALU

C. DNS

D. TLB

3. A process waiting for disk I/O is usually in which state?

A. Running

B. Ready

C. Waiting/blocked

D. New

4. Which algorithm gives each ready process a time quantum?

A. FCFS

B. Round Robin

C. Non-preemptive SJF

D. FIFO I/O

5. CT=20 ms, AT=2 ms and BT=7 ms. What is WT?

A. 11 ms

B. 18 ms

C. 13 ms

D. 9 ms

6. Which algorithm can cause starvation if priorities never change?

A. Priority scheduling

B. FCFS only

C. Round Robin always

D. None

7. What is a race condition?

A. Process termination

B. Outcome depends on concurrent operation order

C. CPU idle

D. Page fault

8. Which is a necessary deadlock condition?

A. Aging

B. Circular wait

C. Time slicing

D. Paging

9. Mutual exclusion means:

A. All processes run together

B. Only one participant enters a protected section at a time

C. No context switches

D. Equal priorities

10. A thread locks a non-recursive mutex it already owns. What can happen?

A. Guaranteed progress

B. Self-deadlock/blocking

C. Automatic unlock

D. Page replacement

Answer key

1-B · 2-A · 3-C · 4-B · 5-A · 6-A · 7-B · 8-B · 9-B · 10-B

Q5 calculation: TAT = CT – AT = 20 – 2 = 18 ms; WT = TAT – BT = 18 – 7 = 11 ms.

Note: This guide uses standard OS explanations to develop the topics identified from the uploaded papers. The practice questions are original revision questions, not verbatim extracts.